VULNERABILITY ANALYSIS & DETECTION

PHP2Shell

CVEs	CVE-2026-109618, CVE-2026-109619
Severity	CVSS v4.0 9.2 (Critical)
Component	PHP engine (session upload progress, out-of-memory shutdown)
Perspective	Blue team, detection engineering
Advisories	GHSA-6wmf-4rv5-p3rp, GHSA-w8hw-w9fx-j79w
Sources	PHP vendor advisories + public analyses (see References)
Author	M3RICK

1 Summary

PHP2Shell is the name given to two memory corruption vulnerabilities in the PHP engine, CVE-2026-109618 and CVE-2026-109619. Both allow an unauthenticated remote attacker to execute code with the privileges of the web server account. The flaws sit in the runtime, not in application code: a server running a correctly written application is still exposed.

Details leaked on 2026-10-20, before coordinated disclosure. Patches were released the next day, exploitation in the wild was reported the same day, and CISA added both CVEs to its Known Exploited Vulnerabilities catalog on 2026-10-26.

CVE	Type	Component
CVE-2026-109619	Use-after-free	Session upload progress (enabled by default)
CVE-2026-109618	Memory corruption	Request shutdown after `memory_limit` is reached

At the time of writing, the published material describes the root causes and the affected versions, and vendors ship scanners that find vulnerable installations. No public source provides indicators of compromise or detection rules for the exploitation itself. This document reconstructs the observable artifacts from the root causes and proposes detection rules for them. It contains no exploit code and does not reproduce the attack.

2 Background: the two vulnerabilities

2.1 CVE-2026-109619: use-after-free from stale request state

The PHP advisory titles this bug *RCE via stale global state* (GHSA-6wmf-4rv5-p3rp) and classes it as a use-after-free (CWE-416). The root cause is not specific to one feature: state owned by a single request outlives the request heap inside a persistent worker, and a later operation that runs before the next request is initialized reads it as if it were valid. A fatal error, out of memory included, can skip the reset that would otherwise clear this state.

The advisory lists more than one trigger. The strongest confirmed case is `fatal_error_backtraces` on PHP 8.5. Session upload progress is a second confirmed trigger, and it is the one with a clear unauthenticated request shape that is enabled by default, so it is the path this document works through and targets for detection. The mechanism below is one instantiation of the stale-state class, not the whole bug.

When `session.upload_progress` is enabled, PHP records the progress of a file upload in the user session while the request body is being received. For speed, it does not look the progress entries up in the session hash table on every chunk: it caches two raw `zval *` pointers directly to them, and writes the updated byte count through those pointers.

The flaw is that the hash table those pointers point into is rebuilt between updates, while the pointers are kept. Reconstructed from the Akamai and watchTowr analyses, the lifecycle is:

1. **First update.** PHP populates the session data hash table with the upload progress entries and caches two `zval *` pointers into it, so later updates can skip the lookup.
2. **Between updates.** To persist progress, PHP flushes the session: it writes the data out, reads it back into a **new** hash table, and frees the old one. The two cached pointers are not refreshed, so they now reference freed `zval` memory.
3. **Second update.** The next chunk arrives after `session.upload_progress.min_freq`. PHP writes the new byte count through the stale pointers. This is a write into freed memory (use-after-free).

4. **Heap grooming.** The large number of repeated form fields in the body is not incidental: each is an allocation the attacker uses to place controlled data into the slot the freed `zval` occupied, so the stale write corrupts a value they chose rather than unused memory.

The second progress update is what arms the bug, which is why the request needs two timed chunks rather than one. A single upload that never triggers a second update writes through pointers that are still valid.

> **Root cause.** The defect is not specific to upload progress. It is an instance of a general antipattern: caching a raw interior pointer into a container, then performing an operation that may reallocate or rebuild that container while treating the cached pointer as still valid. The session flush frees and reconstructs the hash table, which invalidates every pointer into the old allocation, yet the cached `zval *` values survive the operation unchanged. Memory-safe languages reject this construction at compile time. The equivalent in Rust, holding a reference into a `Vec` or `HashMap` across an insertion that triggers reallocation, does not compile, because the borrow checker treats the reallocation as invalidating the borrow. C gives the engine no such guarantee, so the invariant that the cached pointer and the live table refer to the same allocation must be maintained by hand, and here it is not. The corresponding safe pattern is to re-resolve the entry by its key on each update, or to refresh the cached pointers immediately after the table is rebuilt, rather than carrying a pointer across a free.

Two properties make this flaw severe:

- The feature is enabled by default.
- Upload progress is handled while PHP parses the request body, before the target script runs. Any PHP endpoint is reachable, whether or not it expects an upload.

watchTowr notes that upload progress is one confirmed path to this bug, not necessarily the only one. This is consistent with the advisory: the same stale-state fault is reachable through `fatal_error_backtraces` on PHP 8.5, so disabling upload progress narrows the exposure without closing the underlying flaw.

2.2 CVE-2026-109618: memory corruption on out-of-memory

The PHP advisory titles this bug *RCE via OOM corruption* (GHSA-w8hw-w9fx-j79w). When a request reaches `memory_limit`, PHP does not unwind normally. It uses `longjmp()` to jump directly to request shutdown, skipping the cleanup code that would normally run. If this happens while internal request structures are only partly initialized, the shutdown code reads them as if they were valid.

According to Akamai, the shutdown path can then follow a function pointer, such as a destructor, taken from memory the attacker influenced. A large request that exhausts memory at the right moment turns a resource error into control of execution.

3 Attack surface

Item	Detail
Versions	PHP 5.4 to 8.5 before the fixed releases
Fixed in	8.2.35, 8.3.36, 8.4.27, 8.5.12 (source backports exist for 8.1, 8.0, 7.4)
Server APIs	Apache mod_php (highest risk), PHP-FPM, php-cgi, FrankenPHP, LiteSpeed
Authentication	None required
Privileges gained	Those of the web server or PHP worker account

mod_php carries the highest risk because the PHP engine runs inside the Apache worker processes, and the attack completes before any application code runs. Application-level controls (authentication, input validation, a web framework's routing) are never reached.

4 Observable artifacts

This section separates what the sources state from what follows from the root cause. Items marked **inferred** have not been observed in a public incident report.

4.1 Network and request level

The upload progress path requires a specific request shape, described in the Akamai analysis:

- an HTTP `POST` with a `multipart/form-data` body;
- a form field named `PHP_SESSION_UPLOAD_PROGRESS`, which is what turns progress tracking on;
- a session identifier (`PHPSESSID` cookie or field);
- at least one file part, and a pause longer than `session.upload_progress.min_freq` (one second by default) between parts, so that a second progress update takes place;
- an unusually high number of repeated form fields in the same body, used to shape the heap.

> **Inferred.** The pause between parts makes the request slow and the body large relative to what the endpoint normally receives. Request duration and body size outliers on endpoints that never handle uploads are a usable hunting signal.

Standard access logs record neither the body nor the field names. Detecting this request shape requires body inspection: an IDS such as Suricata, a WAF, or ModSecurity audit logging.

4.2 Process level

Memory corruption exploits are not always reliable on the first attempt, and both bugs involve corrupted or exhausted memory. **Inferred:** a failed or partial attempt is likely to kill the PHP worker. This leaves traces in the server error logs:

- PHP-FPM: `child <pid> exited on signal 11 (SIGSEGV)`;
- Apache with mod_php: `AH00052: child pid <pid> exit signal Segmentation fault (11)`;
- either: `Allowed memory size of <n> bytes exhausted`.

One crash proves nothing; PHP extensions crash in production. Several crashes on the same host within a short window are much less common.

A successful attempt ends, per Akamai, in a call to `system()` with an attacker-chosen command. On the host, this appears as a PHP or web server process starting a shell.

4.3 Host level

watchTowr's response guidance lists what to look for after exploitation: new web shells, unexpected cron jobs, new accounts tied to PHP processes, and unexplained outbound connections from PHP processes. Files written to the web root by the web server account outside a deployment are the most direct of these.

5 Detection

Three rules follow from the artifacts above. They target different log sources, so that a gap in one does not blind the whole chain.

Rule	Log source	Purpose
Upload progress POST	IDS (HTTP body)	Hunting: find the request shape
Worker crash burst	PHP-FPM / Apache error log	Failed or noisy attempts
PHP spawns a shell	Process creation (auditd, Sysmon for Linux, EDR)	Successful exploitation

5.1 Suricata: multipart POST with upload progress

Legitimate applications that display an upload progress bar send this field too, so this rule is for hunting rather than alerting. On a server where no application uses upload progress, any match is worth investigating. PHP only tracks progress when the field appears before the file part, so the field sits early in the body and stays within Suricata's default request body inspection limit.

```
# Hunting rule, not a high-confidence alert: legitimate upload progress bars match too.
alert http any any -> $HTTP_SERVERS any (msg:"PHP2Shell hunting: multipart POST with
PHP_SESSION_UPLOAD_PROGRESS"; flow:established,to_server; http.method; content:"POST";
http.content_type; content:"multipart/form-data"; nocase; http.request_body;
content:"PHP_SESSION_UPLOAD_PROGRESS"; nocase; classtype:web-application-attack; sid:9260001;
rev:1;)
```

5.2 Sigma: PHP worker crash burst

A base rule matches individual crash or memory exhaustion lines; a correlation rule raises the severity when they repeat on one host.

```
title: PHP Worker Segfault or Memory Exhaustion
name: php_worker_crash
status: experimental
description: >
  A PHP worker died on SIGSEGV or hit memory_limit. Both are side effects of
  the PHP2Shell memory corruption bugs (CVE-2026-109618, CVE-2026-109619).
  A single event is common noise; see the correlation rule below.
author: M3RICK
date: 2026-10-29
logsource:
  product: linux
  service: php-fpm # custom: php-fpm and Apache error logs
detection:
  keywords:
    - 'exited on signal 11'             # php-fpm master log
    - 'exit signal Segmentation fault'  # Apache mod_php (AH00052)
    - 'Allowed memory size of'          # memory_limit reached
  condition: keywords
level: low
---
title: PHP Worker Crash Burst
status: experimental
description: Repeated PHP worker crashes on the same host in a short window.
author: M3RICK
date: 2026-10-29
correlation:
  type: event_count
  rules:
    - php_worker_crash
  group-by:
    - host
```

```
  timespan: 5m
  condition:
    gte: 5
level: high
```

5.3 Sigma: PHP process spawning a shell

This is the generic web shell pattern and overlaps with existing SigmaHQ rules. It is included because it is the one artifact that confirms code execution, whatever the entry point.

```
title: PHP Process Spawning a Shell or Download Utility
status: experimental
description: >
  A PHP runtime started a shell or a download/network utility. Expected outcome
  of successful PHP2Shell exploitation, and of web shells in general. Overlaps
  with existing SigmaHQ web shell rules; kept here for completeness.
author: M3RICK
date: 2026-10-29
logsource:
  category: process_creation
  product: linux
detection:
  parent:
    ParentImage|contains:
      - '/php-fpm'
      - '/php-cgi'
      - '/lsphp'
      - '/frankenphp'
      - '/apache2'
      - '/httpd'
  child:
    Image|endswith:
      - '/sh'
      - '/bash'
      - '/dash'
      - '/curl'
      - '/wget'
      - '/nc'
      - '/python3'
      - '/perl'
  condition: parent and child
falsepositives:
  - Applications that legitimately call system() or exec() (image processing, mailers)
level: high
```

5.4 Limits

- None of these rules has been tested against real exploitation traffic. They are derived from the published root causes.
- An attacker who succeeds on the first attempt leaves no crash. The process rule is the safety net for that case.
- The process rule depends on process creation logging, which many web servers do not have enabled.

6 Timeline

Date	Event	Source
2026-10-17	Earliest exploitation attempt reported, before disclosure	Amazon TI
2026-10-20	Details leak before coordinated disclosure	watchTowr
2026-10-21	PHP releases 8.2.35, 8.3.36, 8.4.27, 8.5.12	PHP
2026-10-21	Exploitation of both CVEs observed on honeypots	watchTowr
2026-10-23	CVE-2026-115560, a configuration-dependent RCE, disclosed	PHP
2026-10-26	Both CVEs added to the CISA KEV catalog	SOCRadar

7 Mitigation & remediation

- **Patch.** The PHP maintainers state that there is no complete mitigation and that updating is the only safe option.
- **Interim.** Set `session.upload_progress.enabled=0` where upload progress is not used. PHP recommends it for CVE-2026-115560, and it removes the confirmed path to CVE-2026-109619.
- **After patching.** Hunt for the host level artifacts listed above on any server that was exposed between 2026-10-17 and its patch date. A patch closes the hole; it does not remove a web shell already in place.

> **Open question.** Abstract Security argues that the emergency patch fixes the consumer side of CVE-2026-109619 but not the upload progress producer, which remains enabled by default. This is a single source and has not been confirmed elsewhere at the time of writing. Disabling upload progress covers this case either way.

8 Acknowledgements

Drafting, structural editing, and verification of the cited sources were carried out with assistance from Claude (Anthropic). Analysis scope, technical direction, and final review by the author.

9 References

- PHP, *RCE via stale global state* (CVE-2026-109619). https://github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp
- PHP, *RCE via OOM corruption* (CVE-2026-109618). https://github.com/php/php-src/security/advisories/GHSA-w8hw-w9fx-j79w
- Akamai, *Understanding PHP2Shell: Everything You Need to Know*. https://www.akamai.com/blog/security-research/understanding-php2shell-everything-you-need-to-know
- watchTowr, *PHP2Shell RCE Exploited*. https://watchtowr.com/intelligence/php-remote-code-execution-php2shell/
- Qualys, *PHP2Shell: Find, Mitigate, and Patch Now*. https://blog.qualys.com/vulnerabilities-threat-research/2026/10/20/php-announces-critical-security-release-for-two-vulnerabilities
- SOCRadar, *PHP2Shell CVE-2026-109618 & 109619*. https://socradar.io/blog/php2shell-cve-2026-109618-cve-2026-109619/

- Sophos, *Advisory: PHP2Shell Vulnerabilities*. https://www.sophos.com/en-us/security-advisories/sophos-sa-20261021-php2shell
- Abstract Security, *PHP2Shell: Critical PHP RCE Vulnerabilities*. https://www.abstract.security/blog/php2shell-critical-php-rce-vulnerabilities-cve-2026-109618-and-cve-2026-109619